Fachhochschule Nordwestschweiz
Hochschule für Technik

Weight Initialization

Deep Learning

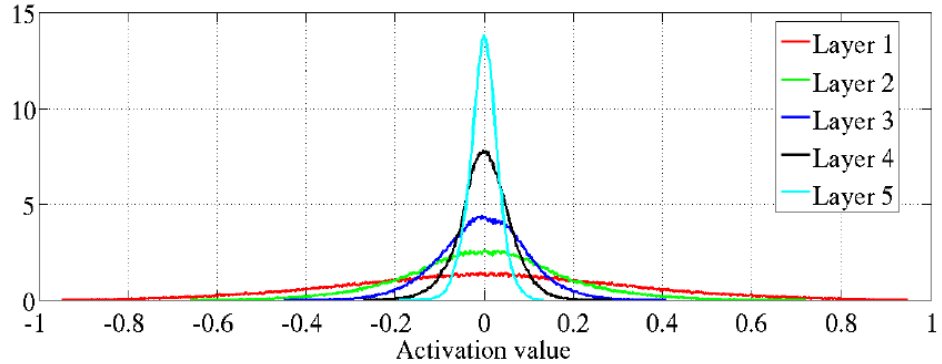
HS 2025

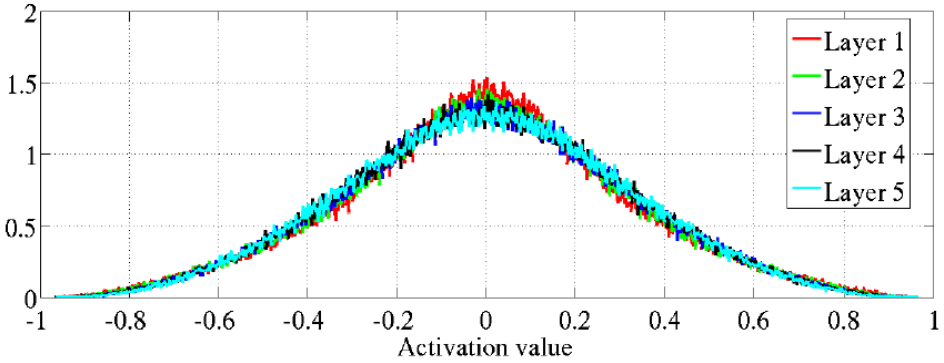
Martin Melchior

Overview

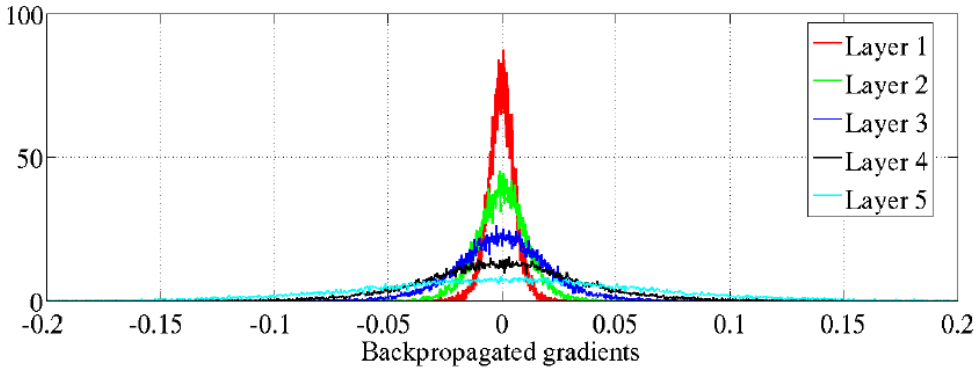
from a paper: X.Glorot & J.Bengio, 2010

Activations

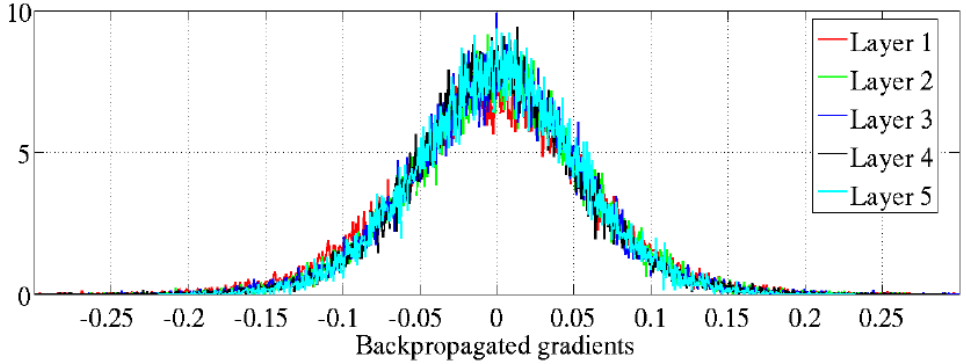




Gradients



Standard normal distributed weights



Suitably scaled normal distributed weights

Model: 5 Layer MLP, trained on Shapeset

Vanishing Gradient Problem

Gradients may vanish in deep neural networks if e.g.

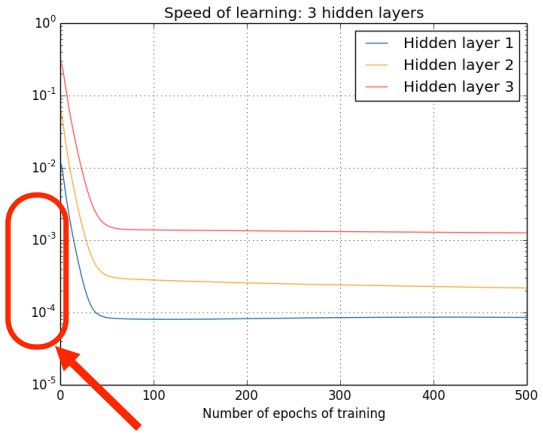
- squeezing activation functions (e.g. Sigmoid) are used
- initial weights are not properly scaled (e.g. just using standard normally distributed).
- no input normalization is applied (model input or layer input)

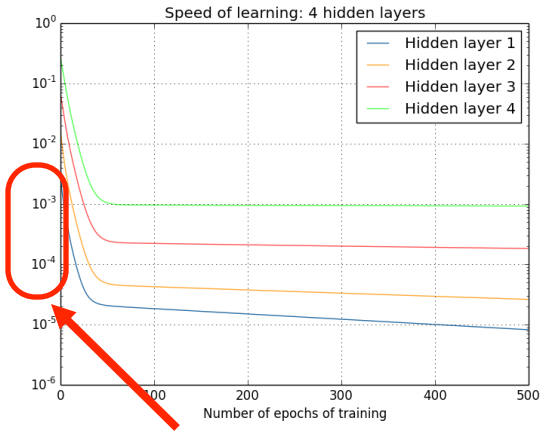
As a result, parameter updates become very small and learning gets stuck.

Vanishing Gradients at Early Layers

Learning MNIST classification with *standard normally distributed initial weights* and sigmoid activations (see neuralnetworksanddeeplearning.com):

Learning is slow in earlier layers and slower the more layers there are.





Note that here, small gradients don't imply that we are close to the optimum!

In other cases (e.g. RNNs) also the opposite can be observed, i.e. exploding gradients.

Complex Dynamics when Training DNNs

- Features in the input data need to be extracted and mapped by the DNN to the label information (in supervised learning).
- Information needs to flow forth (input signal) and back (correction signal) through the DNN over the course of the training.
- At beginning of training,
  - features extracted by earlier layers are random (random weights), contain little information;
  - later layers ignore this information and adapt rather to the signal received from the labels (loss).
- If gradients are small, the signal is hardly propagated back to the early layers and the learning at early layers is slow.

Factors of Influence of Learning Dynamics

driven by layer inputs $x^{[l-1]}$, layer parameters $W^{[l]}, b^{[l]}$

- Distribution of values of the linear affine transformations …
- … should match dynamic range of layer's activation function …
- … both, in forward and backward direction.

$$x^{[l]} = f\left(x^{[l-1]}\left(W^{[l]}\right)^{\mathsf{T}} + b^{[l]}\right)$$

Type of **activation function**, its dynamic range

Normalisation
- input
- output previous layer

Batch Normalisation

Initialisation of Parameters
Weights Initialization

Some Hints on Vanishing Gradients

From backprop:

$$\frac{\partial L}{\partial a^{[l-1]}} = \frac{\partial L}{\partial a^{[l]}} * \boxed{\sigma'(z^{[l]})} \cdot \boxed{W^{[l]}}$$

can be small in saturation regions of activation function

values should not become too small (vanishing gradients) or too large (exploding gradients.

Gradient vanishes exponentially with the depth if the weights are ill-conditioned or the logits are in the saturation region of the activation functions.

Suitable weights initialisation for controlling the variance of the logits and the gradients such that weights evolve at the same rate across layers during training, and no layer reaches the saturation regime before others.

Use non-saturating activation functions as discussed previously.

Analysis of the Gradients

Paper of Glorot and Bengio, 2010 (!)
http://proceedings.mlr.press/v9/glorot10a/glorot10a.pdf

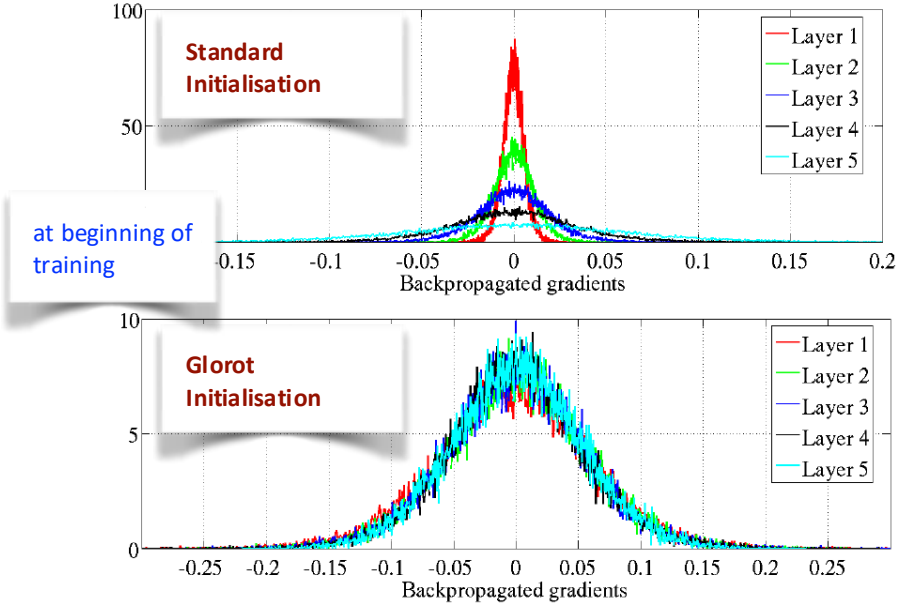


Figure 7: *Back-propagated gradients normalized histograms with hyperbolic tangent activation, with standard (top) vs normalized (bottom) initialization. Top: 0-peak decreases for higher layers.*

Standard initialisation of weights at 2010 (for all layers weights $W^{[l]} \sim \mathcal{N}(0,1)$: Magnitude of gradient components smaller for earlier layers.

Glorot & Bengio could design an initialisation strategy so that the gradient components have similar magnitudes across all layers.

Details: Stabilising Forward Pass

Assume we are in the dynamic range of the activation function, i.e. $\sigma(z) \approx cz$ where c is a constant:

$$a_j^{[l]} = \sigma\left(\sum_k^{N_{l-1}} W_{jk}^{[l]}\, a_k^{[l-1]}\right) \approx c \sum_k^{N_{l-1}} W_{jk}^{[l]}\, a_k^{[l-1]}$$

Assuming all weights in a layer independent, identically distributed and independent from the activations, weights and activations centred:

ignoring indices

$$\begin{aligned} \mathrm{var}\left[a^{[l]}\right] &\approx c\, N_{l-1} \mathrm{var}\left[W^{[l]}\right] \mathrm{var}\left[a^{[l-1]}\right] \\ &= \mathrm{var}\left[a^{[0]}\right] \prod_{j=1}^{l} c\, N_{j-1} \mathrm{var}\left[W^{[j]}\right] \end{aligned}$$

use the following identifies for X,Y independent:
$Var(X+Y) = Var(X) + Var(Y)$
$Var(XY) = Var(X)Var(Y) + (E[X])^2 Var(Y) + Var(X)(E[Y])^2$

Choose:

$$\boxed{\mathrm{var}\left[W^{[j]}\right] = \frac{1}{cN_{j-1}}}$$

"gain" "fan-in"

Details: Stabilising Backward Pass

Similarly, with $\sigma'(z) \approx c$,same assumption as on previous slide:

$$\frac{\partial L}{\partial a_j^{[l-1]}} = \sum_{k=1}^{N_l} \frac{\partial L}{\partial a_k^{[l]}} \sigma'(z_j^{[l]}) W_{kj}^{[l]} \approx c \sum_{k=1}^{N_l} \frac{\partial L}{\partial a_k^{[l]}} W_{kj}^{[l]}$$

and

$$\begin{aligned} \mathrm{var}\left[\frac{\partial L}{\partial a^{[l-1]}}\right] &\approx cN_l \mathrm{var}\left[\frac{\partial L}{\partial a^{[l]}}\right] \mathrm{var}\left[W^{[l]}\right] \\ &= \mathrm{var}\left[\frac{\partial L}{\partial a^{[L]}}\right] \prod_{j=l}^{L} cN_j \mathrm{var}\left[W^{[j]}\right] \end{aligned}$$

Choose: $\boxed{\mathrm{var}\left[W^{[j]}\right] = \frac{1}{cN_j}}$ $\Rightarrow$ $\boxed{\mathrm{var}\left[W^{[j]}\right] = \frac{2/c}{N_{j-1} + N_j}}$

"fan-out"

Compromise of forward and backward pass

Parameter Initialisation

Randomly initialise weights:

- Break symmetries at start of learning.

Initialise weights at proper scale:

- Properly scaling the width of the distribution (depends on the random distribution used to generate the weights and the activation function):

	Activation function	Uniform distribution [–r, r]	Normal distribution
Xavier Glorot	Logistic (sigmoid)	$r = \sqrt{\frac{6}{n_{\text{inputs}} + n_{\text{outputs}}}}$	$\sigma = \sqrt{\frac{2}{n_{\text{inputs}} + n_{\text{outputs}}}}$
Xavier Glorot	Hyperbolic tangent	$r = 4\sqrt{\frac{6}{n_{\text{inputs}} + n_{\text{outputs}}}}$	$\sigma = 4\sqrt{\frac{2}{n_{\text{inputs}} + n_{\text{outputs}}}}$
Kaiming He	ReLU (and its variants)	$r = \sqrt{2}\sqrt{\frac{6}{n_{\text{inputs}} + n_{\text{outputs}}}}$	$\sigma = \sqrt{2}\sqrt{\frac{2}{n_{\text{inputs}} + n_{\text{outputs}}}}$

number of input and output connections **for a given layer**

Remark: Schemes do not guarantee to keep the input/output variance identical in both directions, but provide a good compromise which works well in practice.

Parameter Initialisation PyTorch <-> TF

Different defaults for PyTorch and TF (dense and conv2d layers):

	PyTorch Kaiming He (leaky ReLU)	Tensorflow Xavier Glorot (Tanh)
Weights	$\mathcal{U}(-s,s), s = 1/\sqrt{N_{l-1}}$	$\mathcal{U}(-s,s), s = \sqrt{6/(N_{l-1}+N_l)}$
Bias	$\mathcal{U}(-s,s), s = 1/\sqrt{N_{l-1}}$	0

Parameter Initialisation PyTorch: Implementation Details

Use `torch.nn.init`:

- `xavier_normal_(w)`, `xavier_uniform_(w)`
- `kaiming_normal_(w)`, `kaiming_uniform_(w)`
- `normal_(w)`, `uniform_(w)`

underscore at the end of method name for functions that in-place replace input tensor

```
import math
from torch import Tensor
from torch.nn.init import _calculate_fan_in_and_fan_out, _no_grad_uniform_

def xavier_uniform_(tensor: Tensor, gain: float = 1.) -> Tensor:
    fan_in, fan_out = _calculate_fan_in_and_fan_out(tensor)
    std = gain * math.sqrt(2.0 / float(fan_in + fan_out))
    a = math.sqrt(3.0) * std  # Calculate uniform bounds from standard deviation
    return _no_grad_uniform_(tensor, -a, a)
```

from pytorch source code

compute “units” of given and previous layer

compromise of forward and backward pass

bounds specific for a uniform random variable with given standard deviation

Parameter Initialisation in PyTorch

```
class MLP(torch.nn.Module):
    def __init__(self):
        super(MLP, self).__init__()
        self.flatten = torch.nn.Flatten()
        self.linear1 = torch.nn.Linear(3*32*32, 600)
        self.activ1 = torch.nn.Sigmoid()
        self.linear2 = torch.nn.Linear(600, 10)

    def forward(self, x):
        x = self.flatten(x)
        x = self.linear1(x)
        x = self.activ1(x)
        return self.linear2(x)

model = MLP()

w2 = model.linear2.weight
print(torch.std(w2.view(-1)).item())
torch.nn.init.kaiming_uniform_(w2, mode='fan_in', a=math.sqrt(5))
print(torch.std(w2.view(-1)).item())

torch.nn.init.xavier_normal_(w2)
print(torch.std(w2.view(-1)).item())
```

```
0.023720242083072662
0.023492421954870224
0.0581413134932518
```

`torch.nn.parameter.Parameter`

Replaces in-place parameters values.

By default, initialisation scheme of Kaiming He with leaky ReLu and Uniform distribution; for both, linear and CNN layers.

Customisation of parameter initialisation is easy; typically at model instantiation time.

https://github.com/pytorch/pytorch/blob/main/torch/nn/modules/conv.py

```
class _ConvNd(Module):
    def __init__(

    def reset_parameters(self) -> None:
        # Setting a=sqrt(5) in kaiming_uniform is the same as initializing with
        # uniform(-1/sqrt(k), 1/sqrt(k)), where k = weight.size(1) * prod(*kernel_size)
        # For more details see: https://github.com/pytorch/pytorch/issues/15314#issuecomm
        init.kaiming_uniform_(self.weight, a=math.sqrt(5))
        if self.bias is not None:
```

https://github.com/pytorch/pytorch/blob/main/torch/nn/init.py

```
def kaiming_uniform_(
        return tensor
    fan = _calculate_correct_fan(tensor, mode)
    gain = calculate_gain(nonlinearity, a)
    std = gain / math.sqrt(fan)
    bound = math.sqrt(3.0) * std  # Calculate uniform bounds from standard deviation
    with torch.no_grad():
        return tensor.uniform_(-bound, bound, generator=generator)
```

default mode: “fan_in”

default nonlinearty: “leaky_relu”

```
def _calculate_fan_in_and_fan_out(tensor: Tensor) -> tuple[int, int]:
    dimensions = tensor.dim()
    if dimensions < 2:
        raise ValueError(
            "Fan in and fan out can not be computed for tensor with fewer than 2 dimensions"
        )

    num_input_fmaps = tensor.size(1)
    num_output_fmaps = tensor.size(0)
    receptive_field_size = 1
    if tensor.dim() > 2:
        # math.prod is not always available, accumulate the product manually
        # we could use functools.reduce but that is not supported by TorchScript
        for s in tensor.shape[2:]:
            receptive_field_size *= s
    fan_in = num_input_fmaps * receptive_field_size
    fan_out = num_output_fmaps * receptive_field_size

    return fan_in, fan_out
```